

Full Stack Development with MERN

Project Documentation

Book a Doctor App

1. Introduction

Project Title

Book a Doctor App

Team Members

[Member 1 Name] – Frontend Development, UI Design

[Member 2 Name] – Backend Development, API Integration

[Member 3 Name] – Database Design, Authentication & Testing

[Member 4 Name] – Documentation, Deployment & Maintenance

2. Project Overview

Purpose

The Book a Doctor App is a healthcare appointment booking platform designed to simplify the process of connecting patients with doctors. It enables patients to browse doctors, book appointments, upload medical documents, and manage their healthcare interactions online. The system also allows doctors to manage their schedules and administrators to monitor and control the platform.

The application aims to reduce the time and effort involved in traditional appointment booking by digitizing the entire process. It provides a centralized platform where patients, doctors, and administrators can interact securely and efficiently.

Features

- User registration and login
- Doctor browsing and profile viewing
- Appointment scheduling and cancellation
- Secure medical document upload (reports/prescriptions)
- Doctor dashboard for appointment management
- Admin dashboard for doctor approval and platform monitoring
- Role-based authentication and authorization
- Responsive design for desktop and mobile devices
- Notification support for appointment updates
- Secure password encryption and JWT-based login system

3. Architecture

The Book a Doctor App follows the MERN Stack architecture, where:

- MongoDB is used as the database
- Express.js is used for backend API development
- React.js is used for the frontend user interface
- Node.js is used as the server runtime environment

Frontend

The frontend is developed using React.js and Bootstrap to create a responsive and interactive user interface. React Router is used for page navigation, while Axios is used for communication with backend APIs.

Frontend Responsibilities

- User registration and login forms
- Doctor listing and search pages
- Appointment booking form
- Patient dashboard
- Doctor dashboard
- Admin dashboard
- File upload interface
- Notifications and appointment history display

Backend

The backend is built using Node.js and Express.js. It provides REST APIs for authentication, user management, doctor approval, appointment booking, and document upload.

Backend Responsibilities

- API routing and controller logic
- Authentication using JWT
- Password hashing using bcrypt
- Role-based authorization middleware
- Appointment and doctor management
- File upload handling
- Database communication with MongoDB

Database

MongoDB stores all system data in collections. The main collections are:

- Users
- Doctors
- Appointments
- Reports/Documents (optional collection if stored separately)

MongoDB is chosen because it is flexible, scalable, and well-suited for handling structured and semi-structured healthcare-related data.

4. Setup Instructions

Prerequisites

Before running the project, install the following:

- Node.js (v16 or above)
- npm
- MongoDB or MongoDB Atlas
- Git
- VS Code or any code editor

Installation

Follow these steps to set up the project:

Step 1: Clone the repository

```
git clone https://github.com/your-username/book-a-doctor.git
cd book-a-doctor

cd server
npm install

cd ../client
```

```

npm install

# .env
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret_key

# optional
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

cd server
npm start

cd client
npm start

```

5. Folder Structure

The project is divided into **client** and **server** directories.

Client Structure

```

client/
  public/
  src/
    components/
      Navbar.js
      DoctorCard.js
      AppointmentForm.js
    pages/
      Home.js
      Login.js
      Register.js
      Doctors.js
      Dashboard.js
    services/
      api.js
    App.js
    index.js
  package.json

server/
  config/
  db.js
  controllers/
    authController.js
    doctorController.js
    appointmentController.js
  middleware/
    authMiddleware.js
    roleMiddleware.js
  models/
    User.js
    Doctor.js
    Appointment.js
  routes/
    authRoutes.js
    doctorRoutes.js
    appointmentRoutes.js
  uploads/
  server.js
  package.json
  .env

```

6. Running the Application

To run the complete application locally:

Frontend

```
cd client
npm start
```

Backend

```
cd server
npm start
```

Application Flow

- The backend server runs on `http://localhost:5000`
- The frontend runs on `http://localhost:3000`
- The React app sends API requests to the Express server
- Express processes the requests and communicates with MongoDB
- Responses are returned to the frontend and displayed to users

7. API Documentation

Below are the main APIs used in the project.

1. Authentication APIs

Register User

Endpoint: `/api/auth/register`

Method: POST

Request Body

```
POST /api/auth/register
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "123456",
  "role": "patient"
}
```

```
Response:
{
  "message": "User registered successfully"
}
```

```
POST /api/auth/login
{
  "email": "john@example.com",
  "password": "123456"
}
```

```
Response:
{
  "token": "jwt_token_here",
  "user": {
    "id": "123",
    "name": "John Doe",
    "role": "patient"
  }
}
```

```
GET /api/doctors

[
  {
    "_id": "d1",
    "name": "Dr. Smith",
    "specialization": "Cardiologist",
    "experience": "10 years"
  }
]
```

```
POST /api/appointments/book
{
  "doctorId": "d1",
  "patientId": "u1",
  "date": "2025-03-20",
  "time": "10:30 AM"
}

Response:
{
  "message": "Appointment booked successfully"
}
```

8. Authentication

Authentication and authorization are implemented using JWT (JSON Web Tokens).

Authentication Process

- User registers using name, email, password, and role.
- Password is encrypted using bcrypt before storing in MongoDB.
- On login, the server verifies the email and password.
- If valid, a JWT token is generated and sent to the client.
- The frontend stores the token and includes it in protected API requests.

Authorization

The application uses role-based access control:

- Patient – Can browse doctors, book appointments, upload documents, and manage appointments.
- Doctor – Can view appointment schedules and patient details.
- Admin – Can approve doctors, manage users, and monitor platform activities.

Security Features

- Password hashing using bcrypt
- JWT token verification middleware
- Protected routes for doctors and admins
- Secure file/document upload handling
- Validation of user input before database storage

9. User Interface

The user interface is designed with React.js and Bootstrap to provide a responsive and easy-to-use experience.

Main UI Screens

- Home Page – Introduction and navigation
- Login / Registration Page
- Doctor Listing Page
- Doctor Profile Page
- Appointment Booking Page
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Document Upload Section

UI Features

- Responsive layout for mobile and desktop
- User-friendly forms for booking and login
- Doctor cards with profile information
- Appointment history and booking status
- Dashboard-based access for different roles

Add screenshots here of:

- Home page
- Doctor listing page
- Appointment booking form
- Patient dashboard
- Admin dashboard

10. Testing

Testing ensures that the application works correctly across all modules.

Testing Strategy

The following areas were tested:

- User registration and login
- JWT authentication and role-based access
- Doctor browsing and profile display
- Appointment booking and cancellation
- File upload functionality
- Admin doctor approval workflow
- CRUD operations for users, doctors, and appointments

Tools Used

- Postman – for API testing
- MongoDB Compass – for database verification
- Browser DevTools – for frontend debugging
- Manual testing – for user workflows and UI responsiveness

Validation Checks

- Required fields in forms
- Duplicate email prevention
- Appointment date/time validation
- Invalid login credentials handling
- Token-protected route testing

11. Screenshots or Demo

Include screenshots or demo links here.

Suggested Screenshots

- Home Page
- Registration/Login Page
- Doctors List Page
- Appointment Booking Page
- Patient Dashboard
- Doctor Dashboard
- Admin Dashboard
- Uploaded Documents Page

Demo Link

GitHub Repository: <https://github.com/your-username/book-a-doctor>

Live Demo (if deployed): <https://your-live-demo-link.com>

12. Known Issues

Although the application is functional, some limitations may still exist:

- Real-time doctor availability may not always update instantly without WebSocket integration.
- File upload size and type restrictions may need further enhancement.
- Appointment conflict checking may require stronger validation for overlapping slots.
- Email/SMS notifications may not be fully implemented in the current version.
- The UI may need further accessibility improvements for visually impaired users.

13. Future Enhancements

The following features can be added in future versions:

- Online payment integration for appointment fees
- Video consultation / telemedicine support
- Email and SMS notifications
- Doctor rating and review system
- Advanced search filters by specialization, hospital, and location
- AI-based doctor recommendation system
- Prescription generation and download
- Real-time chat between doctor and patient
- Calendar integration for appointment reminders
- Analytics dashboard for hospital/admin insights

Conclusion

The Book a Doctor App is a full-stack MERN healthcare booking system that simplifies the process of connecting patients with doctors. It provides a secure, efficient, and user-friendly platform for appointment management, doctor browsing, and administrative control. Through this project, important concepts such as React frontend development, Express backend APIs, MongoDB database design, JWT authentication, role-based access control, and full CRUD operations are implemented in a practical real-world application.

This project demonstrates how modern web technologies can be used to improve healthcare accessibility and management by digitizing appointment booking and communication between patients, doctors, and administrators.